Software Requirements Specification

for

Ride Booking System

Version 1.0 approved

Prepared by Akshay Talpade

Videsh Tambe

Sardar Patel Institute of Technology,Andheri

27/07/26

Copyright © 1999 by Karl E. Wiegers. Permission is granted to use, modify, and distribute this document.

Table of Contents

Table of Contents ii
Revision History ii
1. Introduction 1
1.1 Purpose 1
1.2 Document Conventions 1
1.3 Intended Audience and Reading Suggestions 1
1.4 Product Scope 1
1.5 References 1
2. Overall Description 2
2.1 Product Perspective 2
2.2 Product Functions 2
2.3 User Classes and Characteristics 2
2.4 Operating Environment 2
2.5 Design and Implementation Constraints 2
2.6 User Documentation 2
2.7 Assumptions and Dependencies 3
3. External Interface Requirements 3
3.1 User Interfaces 3
3.2 Hardware Interfaces 3
3.3 Software Interfaces 3
3.4 Communications Interfaces 3
4. System Features 4
4.1 System Feature 1 4
4.2 System Feature 2 (and so on) 4
5. Other Nonfunctional Requirements 4
5.1 Performance Requirements 4
5.2 Safety Requirements 5
5.3 Security Requirements 5
5.4 Software Quality Attributes 5
5.5 Business Rules 5
6. Other Requirements 5
Appendix A: Glossary 5
Appendix B: Analysis Models 5
Appendix C: To Be Determined List 6

Revision History

Name	Date	Reason For Changes	Version

1. Introduction

The Software Requirements Specification (SRS) document provides a detailed description of the Ride Booking System and specifies the functional and non-functional requirements that the system must satisfy. This document serves as a formal agreement between the project stakeholders, developers, testers, and future maintainers by clearly defining the expected behavior, constraints, interfaces, and performance of the software. The SRS acts as a foundation for system design, implementation, testing, deployment, and maintenance throughout the software development lifecycle.

The Ride Booking System is designed to provide a convenient, efficient, and secure platform for connecting passengers with nearby drivers through an online application. It simplifies the traditional ride-booking process by enabling users to request rides, track drivers in real time, make secure payments, and manage trip histories digitally. The system also provides a dedicated interface for drivers to manage ride requests and earnings, while administrators oversee user management, driver verification, and overall system operations.

1.1 Purpose

The primary purpose of this Software Requirements Specification (SRS) is to define the complete set of software requirements for the Ride Booking System. It identifies the expected functionalities, external interfaces, constraints, assumptions, and quality attributes required for successful development and deployment of the application.

The Ride Booking System aims to eliminate the inconvenience associated with traditional transportation booking methods by providing an automated and intelligent platform for ride management. Through the system, passengers can easily register, search for nearby rides, view fare estimates, book rides, make online payments, and monitor their trips in real time. Drivers can receive ride requests, accept or reject bookings, navigate to passenger locations, and monitor their earnings through a dedicated dashboard. Administrators can supervise the entire system by managing users, verifying driver documents, monitoring ongoing rides, resolving complaints, and generating reports.

This document acts as a reference throughout the software development lifecycle and helps ensure that all stakeholders share a common understanding of the project requirements before implementation begins.

The objectives of this document are:

- To define all functional requirements of the Ride Booking System.

- To specify non-functional requirements such as security, reliability, scalability, and performance.
- To describe the interaction between users and the system.
- To provide guidelines for developers during implementation.
- To assist testers in preparing software test cases.
- To serve as a reference for future system maintenance and enhancements.

1.2 Document Conventions

This Software Requirements Specification follows the IEEE 29148 Software Requirements Specification standard. The document is organized into numbered sections and subsections to improve readability and maintain consistency throughout the project documentation.

The following conventions are used in this document:

- Main chapters are numbered sequentially (e.g., 1, 2, 3).
- Subsections are numbered hierarchically (e.g., 1.1, 2.3, 4.1.2).
- Functional requirements are uniquely identified using requirement identifiers such as **REQ-001**, **REQ-002**, and **REQ-003**.
- Requirement priorities are categorized as **High**, **Medium**, or **Low**, depending on their importance to the system.
- Technical terms and abbreviations are defined in the Glossary section for better understanding.
- All requirements are written using the word **"shall"** wherever applicable to indicate mandatory system behavior, following IEEE standards.

The document uses formal and precise language to eliminate ambiguity and ensure that each requirement can be verified during testing

1.3 Intended Audience and Reading Suggestions

This document is intended for all individuals involved in the development, testing, deployment, maintenance, and evaluation of the Ride Booking System.

The intended audience includes:

Project Guide

The project guide reviews the document to ensure that the proposed system satisfies academic objectives and follows software engineering standards. The guide also verifies the completeness and feasibility of the specified requirements.

Software Developers

Developers use this document as the primary reference during system design and implementation. It provides detailed descriptions of system functionalities, interfaces, constraints, and business rules that guide software development.

Software Testers

Testing teams utilize the requirements defined in this document to prepare unit tests, integration tests, system tests, and acceptance tests. Each functional requirement serves as the basis for validating system behavior.

Project Team Members

All project members refer to this document to understand project objectives, user requirements, workflows, and implementation expectations. It ensures consistency among all members during development.

System Administrators

Administrators responsible for deployment and maintenance use this document to understand operational requirements, security measures, backup procedures, and system constraints.

Future Maintenance Team

Future developers and maintenance engineers use this document to understand the existing architecture and system requirements before introducing new features or modifications.

Reading Suggestions

Readers are advised to follow the document in the following sequence:

1. Read the **Introduction** to understand the project's objectives.
2. Review the **Overall Description** for an overview of the system.
3. Study the **External Interface Requirements** to understand interactions with users and external systems.
4. Examine the **System Features** section to understand detailed functional requirements.
5. Review the **Non-Functional Requirements** for quality expectations.
6. Finally, refer to the appendices for diagrams, glossary, and supporting information.

1.4 Product Scope

The Ride Booking System is an online transportation management application that connects passengers with nearby drivers using location-based services and modern communication technologies. The system provides a centralized platform where users can request rides, drivers can accept ride requests, and administrators can monitor and manage overall system activities.

The primary objective of the application is to provide a safe, reliable, and convenient transportation service while minimizing manual intervention. By automating ride allocation, fare estimation, payment processing, and ride tracking, the system significantly improves operational efficiency and user satisfaction.

The major features of the Ride Booking System include:

- User Registration and Authentication
- Driver Registration and Verification
- Ride Booking
- Ride Cancellation
- Automatic Driver Assignment
- Real-Time GPS Tracking
- Fare Estimation
- Multiple Payment Options
- Digital Invoice Generation
- Ride History
- Rating and Review System
- Push Notifications
- Admin Dashboard
- Driver Earnings Management
- Complaint Management

The expected benefits of the system include:

- Reduced waiting time for passengers.
- Efficient ride allocation using location-based services.
- Improved transparency in fare calculation.
- Secure digital payment processing.
- Better driver management.
- Real-time tracking for enhanced passenger safety.
- Simplified transportation management through centralized administration.

The system is scalable and can be extended in the future by integrating advanced technologies such as artificial intelligence for ride prediction, dynamic pricing algorithms, voice-based booking, electric vehicle support, and predictive demand analysis.

1.5 References

The development of the Ride Booking System is based on various software engineering standards, technical documentation, and software development resources. The following references provide guidelines and technical support during system development:

1. IEEE Std 29148-2018 – Systems and Software Engineering – Life Cycle Processes – Requirements Engineering.
2. Karl E. Wiegers, *Software Requirements*, Microsoft Press.
3. Oracle Java Documentation for Java Programming Language.

4. Spring Boot Official Documentation.
5. MySQL Official Documentation.
6. Google Maps Platform API Documentation.
7. REST API Design Guidelines.
8. JSON (JavaScript Object Notation) Standard Documentation.
9. Payment Gateway API Documentation (Razorpay/Stripe or equivalent).
10. HTML5, CSS3, and JavaScript Documentation.
11. React.js Documentation (if React is used for frontend development).
12. Software Engineering principles and guidelines prescribed by the Department of Computer Engineering, Sardar Patel Institute of Technology (SPIT).

The above references serve as the foundation for requirement specification, software architecture, implementation, testing, and future maintenance of the Ride Booking System.

2. Overall Description

The Ride Booking System is a location-based transportation platform that enables passengers to book rides conveniently through a web or mobile application. The system acts as an intermediary between riders and drivers by automatically matching passengers with nearby available drivers based on their current location. It simplifies the process of requesting transportation by integrating GPS navigation, fare estimation, secure online payment, and real-time ride tracking into a single platform.

The system is designed to improve the overall transportation experience by reducing waiting time, increasing operational efficiency, and ensuring secure communication between all users. It supports three major stakeholders: Riders, Drivers, and Administrators. Each stakeholder has dedicated functionalities and access privileges that ensure smooth operation of the application.

The Ride Booking System aims to replace traditional methods of booking rides with an intelligent and automated solution that offers convenience, transparency, reliability, and safety.

2.1 Product Perspective

The Ride Booking System is a standalone application developed to provide transportation services through digital platforms. It is a completely new software product that integrates multiple technologies such as GPS services, online payment gateways, cloud databases, and web APIs to automate ride booking and management.

The system follows a client-server architecture where users interact with the application through a graphical user interface while the backend server processes requests, stores data, and communicates with external services. All ride requests, payment transactions, driver assignments, and notifications are handled centrally by the application server.

The system consists of the following major modules:

Rider Module

The Rider Module allows passengers to register, log in, search for rides, book rides, view fare estimates, make payments, track drivers in real time, manage their ride history, and submit ratings and reviews after completing a trip.

Driver Module

The Driver Module enables drivers to register, upload vehicle documents, receive ride requests, accept or reject bookings, navigate to passenger locations using GPS, complete rides, monitor earnings, and maintain their profiles.

Administrator Module

The Administrator Module provides complete control over system operations. Administrators can verify driver registrations, manage users, monitor ride activities, resolve complaints, oversee payment records, and generate analytical reports.

Database Module

The database stores all application data including user profiles, driver information, vehicle details, ride records, payment transactions, ratings, notifications, and administrative logs.

External Services

The application communicates with external services such as:

- Google Maps API for location and navigation
- Payment Gateway for secure transactions
- SMS/Email Notification Service
- GPS Services

These external services improve the overall functionality and user experience of the application.

2.2 Product Functions

The Ride Booking System provides several core functionalities that collectively automate the complete ride booking process. The major functions of the system are described below.

User Registration and Login

The system allows new users to create an account using personal information such as name, email address, phone number, and password. Registered users can securely log in using authenticated credentials.

Driver Registration

Drivers can register themselves by providing personal information, driving license details, vehicle information, and other required documents. Driver accounts become active only after administrator verification.

Ride Booking

Passengers can book rides by selecting pickup and destination locations. The system automatically estimates the fare and assigns the nearest available driver.

Fare Estimation

The application calculates an estimated fare based on factors such as travel distance, estimated travel time, ride category, and applicable pricing rules.

Driver Matching

The system identifies nearby available drivers using GPS data and assigns the most suitable driver based on proximity and availability.

Real-Time Ride Tracking

Passengers can track the driver's location before pickup and monitor the trip in real time until completion.

Online Payment

The application supports multiple payment methods including UPI, debit cards, credit cards, digital wallets, and cash payments. Secure payment processing ensures safe financial transactions.

Ride History

Users can access detailed information about previous rides, including travel routes, fare details, payment status, and driver information.

Rating and Review System

Passengers can provide ratings and reviews for completed rides, helping maintain service quality and accountability.

Notifications

The system sends real-time notifications regarding ride confirmation, driver arrival, ride completion, payment status, and promotional updates.

Administration

Administrators manage users, verify drivers, monitor ongoing rides, generate reports, and handle complaints to ensure efficient system operation.

2.3 User Classes and Characteristics

The Ride Booking System serves different categories of users, each having unique roles, responsibilities, and access privileges.

Rider

The Rider is the primary user of the application. Riders use the system to request transportation services and monitor ride progress.

Characteristics:

- Basic smartphone knowledge
- Internet connectivity
- GPS-enabled device
- Registered account

Responsibilities:

- Register and log in
- Search for rides
- Book rides
- Cancel rides before pickup
- Track drivers
- Make payments
- Rate completed rides
- View ride history

Driver

Drivers provide transportation services through the application.

Characteristics:

- Valid driving license
- Registered vehicle
- Smartphone with GPS
- Internet connection
- Administrator-approved account

Responsibilities:

- Accept or reject ride requests
- Navigate to pickup locations
- Complete rides
- Receive payments
- View earnings
- Update availability status

Administrator

The administrator manages the overall functioning of the Ride Booking System.

Characteristics:

- Technical knowledge
- Administrative privileges
- Secure login credentials

Responsibilities:

- Verify driver documents
- Manage user accounts
- Monitor ride activity
- Handle complaints
- Generate reports
- Suspend fraudulent accounts
- Maintain system security

2.4 Operating Environment

The Ride Booking System operates on multiple software and hardware platforms to ensure accessibility and reliability.

Client Environment

Supported Operating Systems:

- Windows 10 and above
- Android 10 and above
- Ubuntu Linux

Supported Browsers:

- Google Chrome
- Mozilla Firefox
- Microsoft Edge

Minimum Hardware Requirements:

- Dual-Core Processor
- 4 GB RAM
- GPS-enabled smartphone
- Stable internet connection

Server Environment

Backend Framework:

- Spring Boot (Java)

Database:

- MySQL Server

Operating System:

- Ubuntu Linux or Windows Server

Minimum Server Specifications:

- Quad-Core Processor
- 8 GB RAM
- SSD Storage
- Reliable broadband connection

External Components

The application requires integration with:

- Google Maps Platform
- Payment Gateway
- Email Service

- SMS Notification Service

2.5 Design and Implementation Constraints

The development and operation of the Ride Booking System are subject to several constraints that influence design decisions and implementation strategies.

Technical Constraints

- Internet connectivity is mandatory.
- GPS must remain enabled during active rides.
- Google Maps API requires a valid API key.
- Payment gateway services depend on third-party providers.
- The application must support multiple devices and screen sizes.

Development Constraints

- Backend shall be developed using Java Spring Boot.
- MySQL shall be used as the relational database.
- RESTful APIs shall be used for communication.
- Secure authentication mechanisms shall be implemented.

Security Constraints

- Passwords must be encrypted.
- HTTPS shall be used for secure communication.
- Only verified drivers may accept ride requests.
- Payment information shall never be stored in plain text.

Business Constraints

- Drivers cannot accept multiple rides simultaneously.
- Users cannot have more than one active ride at a time.
- Driver verification is mandatory before activation.

2.6 User Documentation

The Ride Booking System will provide comprehensive documentation for all categories of users.

The documentation includes:

User Manual

Provides instructions for registration, login, booking rides, making payments, tracking rides, and managing profiles.

Driver Manual

Explains driver registration, document upload, ride acceptance, navigation, earnings management, and profile maintenance.

Administrator Manual

Describes administrative operations including user management, driver verification, report generation, complaint handling, and system monitoring.

Installation Guide

Provides instructions for deploying the application on supported platforms.

Online Help

Integrated help pages and FAQs assist users in resolving common issues.

2.7 Assumptions and Dependencies

The successful operation of the Ride Booking System depends upon several assumptions and external services.

Assumptions

- Users possess smartphones with internet access.
- GPS services remain enabled during rides.
- Drivers maintain updated vehicle documents.
- Payment gateway services remain operational.
- Google Maps API services are continuously available.

Dependencies

The application depends on the following external systems:

Google Maps API

Provides mapping, routing, navigation, geolocation, and distance calculations.

Payment Gateway

Processes secure online financial transactions.

Internet Connectivity

Required for ride booking, live tracking, payment processing, and communication.

Email Service

Used for account verification, password recovery, invoices, and notifications.

SMS Service

Delivers OTP verification, ride confirmation, and emergency notifications.

Database Server

Stores all application data securely and ensures data consistency.

Future Scalability

The Ride Booking System is designed with scalability in mind. Future enhancements may include:

- AI-based driver allocation
- Dynamic fare pricing
- Electric vehicle support
- Ride-sharing (Carpool)
- Voice-based booking
- Multi-language support
- Predictive demand analysis
- AI-powered customer support chatbot
- Integration with wearable devices
- Advanced analytics dashboard

These enhancements can be incorporated without significantly modifying the existing system architecture, ensuring long-term maintainability and extensibility.

3. External Interface Requirements

The External Interface Requirements describe how the Ride Booking System interacts with its users, hardware devices, software applications, and communication networks. These interfaces define the communication between different system components and ensure that users can access the application efficiently while maintaining security, reliability, and usability.

The Ride Booking System provides a responsive and user-friendly interface for passengers, drivers, and administrators. It also integrates with external services such as Google Maps, GPS modules, payment gateways, email services, and notification services to provide a seamless ride-booking experience.

The following sections describe the various interfaces required by the system.

3.1 User Interfaces

The Ride Booking System provides separate user interfaces for Riders, Drivers, and Administrators. The interfaces are designed using modern UI/UX principles to ensure ease of use, accessibility, consistency, and responsiveness across various devices.

The application follows a responsive web design that adapts to different screen sizes, including desktops, tablets, and smartphones. The user interface uses a clean layout, intuitive navigation, meaningful icons, readable typography, and consistent color schemes to improve user experience.

3.1.1 Rider Interface

The Rider Interface enables passengers to book rides, track drivers, manage payments, and view previous trips.

Registration Page

The registration page allows new users to create an account by providing the following details:

- Full Name
- Mobile Number
- Email Address
- Password
- Confirm Password

The system validates all fields before creating an account. Invalid email formats, duplicate mobile numbers, or weak passwords generate appropriate error messages.

Login Page

Registered users can securely log into the system using their registered email address or mobile number and password.

Features include:

- Remember Me
- Forgot Password
- OTP Verification (optional)

- Secure Authentication

Home Dashboard

The home dashboard is the primary screen displayed after login. It contains:

- Current Location
- Search Bar
- Ride Categories
- Recent Locations
- Saved Addresses
- Navigation Menu
- User Profile

Ride Booking Screen

Passengers can:

- Select Pickup Location
- Select Destination
- View Estimated Fare
- Choose Ride Type
- Confirm Booking

The system displays available drivers nearby before confirming the ride.

Live Tracking Screen

After ride confirmation, users can view:

- Driver's Live Location
- Driver Name
- Vehicle Number
- Estimated Arrival Time (ETA)
- Route Map
- Ride Status

The location updates automatically at regular intervals.

Payment Screen

Users can complete payments using:

- UPI
- Debit Card

- Credit Card
- Digital Wallet
- Cash

The system generates a digital receipt after successful payment.

Ride History Screen

Passengers can access:

- Previous Trips
- Fare Details
- Payment Status
- Driver Information
- Trip Duration
- Invoice Download

Rating and Review Screen

Once the ride is completed, users can:

- Provide Star Ratings (1–5)
- Submit Written Reviews
- Report Issues
- Submit Complaints

3.1.2 Driver Interface

The Driver Interface enables drivers to manage ride requests and monitor their work.

The dashboard displays:

- Driver Profile
- Online/Offline Status
- Current Ride
- Upcoming Ride Requests
- Daily Earnings
- Weekly Earnings
- Monthly Earnings

Incoming Ride Requests

Each request displays:

- Passenger Name
- Pickup Location
- Destination
- Estimated Fare
- Distance to Pickup
- Estimated Trip Time

Drivers can either:

- Accept Ride
- Reject Ride

Accepted rides automatically start navigation using Google Maps.

Earnings Dashboard

Drivers can monitor:

- Total Earnings
- Completed Trips
- Bonus Amount
- Incentives
- Payment History

3.1.3 Administrator Interface

The Administrator Interface provides complete control over system operations.

Major modules include:

User Management

Administrators can:

- Add Users
- Delete Users
- Suspend Accounts
- Reset Passwords

Driver Verification

Administrators verify:

- Driving License
- Vehicle Registration
- Identity Proof
- Insurance Documents

Only verified drivers can accept ride requests.

Ride Monitoring

Administrators can monitor:

- Active Rides
- Completed Rides
- Cancelled Rides
- Ride Statistics

Report Dashboard

The dashboard provides graphical reports such as:

- Daily Revenue
- Monthly Revenue
- Number of Active Users
- Total Drivers
- Ride Completion Rate
- Customer Satisfaction

User Interface Design Standards

The application follows standard design principles including:

- Responsive Design
- Consistent Navigation
- Uniform Color Scheme
- Easy Accessibility
- Readable Typography
- Fast Loading Screens
- Interactive Maps
- Standard Error Messages
- User-Friendly Forms

3.2 Hardware Interfaces

The Ride Booking System communicates with several hardware devices that enable location tracking, internet connectivity, and application execution.

Mobile Devices

The application supports Android smartphones and desktop systems.

Recommended minimum hardware:

- Dual-Core Processor
- 4 GB RAM
- GPS Support
- Internet Connectivity
- Touchscreen Display

GPS Module

GPS hardware is essential for:

- Detecting User Location
- Detecting Driver Location
- Live Ride Tracking
- Route Navigation
- Distance Calculation
- ETA Calculation

The GPS coordinates are continuously updated during an active ride.

Internet Connectivity

A stable internet connection is mandatory for:

- Login
- Registration
- Ride Booking

- Driver Assignment
- GPS Tracking
- Payment Processing
- Notification Delivery

Supported networks include:

- Wi-Fi
- 4G
- 5G
- Broadband

Server Hardware

The application server performs:

- User Authentication
- Database Processing
- Ride Assignment
- Payment Processing
- Notification Delivery

Recommended server configuration:

- Quad-Core Processor
- 8 GB RAM
- SSD Storage
- Linux Server
- High-Speed Internet

3.3 Software Interfaces

The Ride Booking System interacts with multiple software components and external APIs.

Operating System

The application supports:

- Windows 10/11

- Ubuntu Linux
- Android

Database Interface

The system uses **MySQL** as the backend database.

The database stores:

- User Details
- Driver Information
- Vehicle Details
- Booking Records
- Ride History
- Payments
- Ratings
- Complaints
- Notifications

The backend communicates with the database through SQL queries.

Backend Framework

The backend is developed using **Spring Boot (Java)**.

Responsibilities include:

- User Authentication
- Ride Allocation
- Payment Processing
- API Management
- Database Connectivity
- Business Logic Implementation

Frontend Framework

The application frontend is developed using:

- HTML5
- CSS3
- JavaScript
- React.js (or another suitable framework)

The frontend provides an interactive and responsive user experience.

Google Maps Platform API

Google Maps API is integrated to provide:

- Interactive Maps
- Location Search
- Route Planning
- Distance Calculation
- Live Navigation
- Reverse Geocoding
- Estimated Arrival Time

The API enables accurate navigation and driver tracking.

Payment Gateway

The system integrates with secure payment gateways such as Razorpay or Stripe to process digital transactions.

Supported payment methods include:

- UPI
- Credit Cards
- Debit Cards
- Net Banking
- Digital Wallets

The payment gateway communicates securely using encrypted API requests.

Notification Services

The application integrates notification services for sending:

- Ride Confirmation
- Driver Arrival Alerts
- Ride Completion Messages
- Payment Confirmation
- Promotional Notifications
- Emergency Alerts

Notifications can be delivered through:

- Push Notifications
- SMS
- Email

3.4 Communications Interfaces

The Ride Booking System communicates securely over the internet using standard networking protocols.

HTTP and HTTPS Protocols

The client and server communicate using HTTP and HTTPS protocols.

HTTPS is mandatory for protecting sensitive user information, payment details, and authentication credentials through SSL/TLS encryption.

RESTful APIs

The application uses REST APIs for communication between the frontend and backend.

The APIs support operations such as:

- User Registration
- Login Authentication
- Ride Booking
- Ride Cancellation
- Driver Assignment
- Payment Processing
- Ride Tracking
- Rating Submission
- Profile Management

API responses are exchanged in JSON format.

JSON Data Exchange

The application uses **JSON (JavaScript Object Notation)** as the standard data format for communication between the client application and server.

Typical JSON data includes:

- User Information
- Driver Details
- Ride Details
- Payment Status
- Notification Messages

JSON provides a lightweight and platform-independent communication mechanism.

Network Requirements

The application requires:

- Stable Internet Connection
- Minimum 1 Mbps Bandwidth
- Low Network Latency
- Continuous GPS Connectivity

If network connectivity is lost during a ride, the system attempts to reconnect automatically and synchronize ride data once the connection is restored.

Security in Communication

To ensure secure communication between all system components, the Ride Booking System implements the following security measures:

- HTTPS Encryption
- SSL/TLS Certificates
- JWT-Based Authentication
- Secure API Tokens
- Session Timeout
- Encrypted Password Storage
- Input Validation
- SQL Injection Prevention

- Cross-Site Scripting (XSS) Protection
- Cross-Site Request Forgery (CSRF) Protection

These security mechanisms protect user data, payment information, and communication channels from unauthorized access and cyber threats.

4. System Features

This section describes the major functional features of the Ride Booking System. Each feature includes a description, priority level, stimulus/response sequence, and detailed functional requirements. These requirements specify how the system should behave under different conditions and interactions initiated by users, drivers, or administrators.

4.1 User Registration

4.1.1 Description and Priority

The User Registration feature enables new passengers to create an account in the Ride Booking System. Registration is required before users can access ride-booking services. The system collects essential user information such as full name, email address, mobile number, and password. Before creating the account, the system validates all entered details to ensure correctness and uniqueness.

This feature ensures secure onboarding of new users while maintaining the integrity of stored data.

Priority: **High**

4.1.2 Stimulus/Response Sequences

User Action	System Response
User opens registration page	Registration form is displayed
User enters personal information	System validates all input fields
User clicks "Register"	System checks email and mobile uniqueness
Information is valid	Account is created successfully
Information is invalid	Appropriate error message is displayed
Registration completed	User is redirected to Login page

4.1.3 Functional Requirements

REQ-001

The system shall allow new users to register using their personal details.

REQ-002

The system shall validate email format before account creation.

REQ-003

The system shall verify that the mobile number is unique.

REQ-004

The system shall encrypt user passwords before storing them.

REQ-005

The system shall notify the user when registration is successful.

REQ-006

The system shall display meaningful validation messages whenever incorrect information is entered.

4.2 User Login and Authentication

4.2.1 Description and Priority

Description:

The Login feature authenticates registered users before granting access to the Ride Booking System. Authentication ensures that only authorized users can access their accounts and personal information.

The system securely verifies login credentials against stored user records and establishes a secure user session upon successful authentication.

Priority: High

4.2.2 Stimulus/Response Sequences

User Action	System Response
User enters login credentials	System validates credentials
Credentials are correct	Dashboard is displayed
Password is incorrect	Error message shown
User selects Forgot Password	Password recovery process starts

4.2.3 Functional Requirements

REQ-007

The system shall authenticate registered users.

REQ-008

The system shall reject invalid login credentials.

REQ-009

The system shall provide password recovery functionality.

REQ-010

The system shall maintain secure login sessions.

REQ-011

The system shall automatically log out inactive users after a specified timeout period.

4.3 Driver Registration and Verification

4.3.1 Description and Priority

Description:

Drivers must register before providing transportation services. During registration, drivers submit personal information, driving license details, vehicle information, insurance details, and identity proof.

The administrator verifies all submitted documents before activating the driver's account.

This process ensures that only verified drivers participate in the platform, thereby improving passenger safety.

Priority: High

4.3.2 Stimulus/Response Sequences

Driver Action	System Response
Driver submits registration form	System stores information
Driver uploads documents	Documents are forwarded for verification
Administrator verifies documents	Driver account becomes active
Documents rejected	Driver receives notification

4.3.3 Functional Requirements

REQ-012

The system shall allow drivers to register.

REQ-013

The system shall allow uploading of driving license.

REQ-014

The system shall allow uploading vehicle registration documents.

REQ-015

The administrator shall verify all submitted documents.

REQ-016

Only verified drivers shall be allowed to accept ride requests.

REQ-017

The system shall notify drivers regarding verification status.

4.4 Ride Booking

4.4.1 Description and Priority

Description:

Ride Booking is the core functionality of the Ride Booking System. Registered passengers can request transportation by selecting pickup and destination locations. The system estimates the fare, identifies nearby available drivers, and assigns the most suitable driver.

The booking process is completed within a few seconds, providing a seamless user experience.

Priority: High

4.4.2 Stimulus/Response Sequences

User Action	System Response
User selects pickup location	System identifies current location
User enters destination	Distance is calculated
User confirms booking	Nearby drivers are searched
Driver accepts request	Ride confirmation sent
Driver unavailable	Alternative driver searched

4.4.3 Functional Requirements

REQ-018

The system shall allow passengers to enter pickup locations.

REQ-019

The system shall allow passengers to enter destination locations.

REQ-020

The system shall calculate estimated travel distance.

REQ-021

The system shall estimate fare before booking confirmation.

REQ-022

The system shall assign the nearest available driver.

REQ-023

The system shall notify both rider and driver after successful booking.

4.5 Ride Cancellation

4.5.1 Description and Priority

Description:

Passengers or drivers may cancel rides before the trip begins under permitted conditions. The system processes cancellations, updates ride status, informs the concerned party, and applies cancellation policies where applicable.

This feature ensures flexibility while minimizing inconvenience for both passengers and drivers.

Priority: Medium

4.5.2 Stimulus/Response Sequences

User Action	System Response
Rider requests cancellation	Ride status updated
Driver receives cancellation notification	Ride removed from active list
Cancellation policy applicable	Charges calculated
Cancellation successful	Confirmation displayed

4.5.3 Functional Requirements

REQ-024

The system shall allow passengers to cancel rides before pickup.

REQ-025

The system shall notify drivers immediately after cancellation.

REQ-026

The system shall update ride status in the database.

REQ-027

The system shall apply cancellation charges according to predefined business rules.

REQ-028

The system shall record cancellation history.

5. Other Nonfunctional Requirements

5.1 Performance Requirements

The Ride Booking System shall provide fast, efficient, and reliable services even during peak hours when multiple users are accessing the application simultaneously. Performance is a critical factor because users expect immediate responses while booking rides, tracking drivers, and making payments.

The system shall be capable of handling concurrent requests without significant degradation in response time. Ride requests, driver assignments, and payment confirmations should be processed efficiently to provide a smooth user experience.

The application shall support at least **10,000 concurrent users** without affecting the stability of the system. Response time for common operations such as user login, ride booking, fare estimation, and ride cancellation shall not exceed **2–3 seconds** under normal operating conditions.

The database shall efficiently process read and write operations to minimize delays during data retrieval and storage. Frequently accessed data such as user profiles, ride history, and fare calculations should be optimized using indexing and caching techniques.

The system shall automatically recover from temporary failures and continue processing user requests with minimal interruption.

Performance Requirements

- Response time for login shall be less than **2 seconds**.
- Ride booking confirmation shall be completed within **5 seconds**.
- Driver matching shall occur within **10 seconds**.
- Live GPS location shall refresh every **5–10 seconds**.
- Payment confirmation shall be displayed within **5 seconds**.
- The system shall maintain an uptime of **99.9%**.
- Database queries shall execute efficiently under normal load conditions.

5.2 Safety Requirements

Passenger and driver safety is one of the most important objectives of the Ride Booking System. The application shall implement several safety mechanisms to protect users during every stage of the ride.

The system shall verify every driver's identity and vehicle documents before allowing them to provide transportation services. Unverified drivers shall not be allowed to accept ride requests.

During an active trip, passengers shall have access to real-time driver location, vehicle details, and estimated arrival time. The application should provide an emergency contact or SOS feature that enables passengers to quickly seek assistance during emergencies.

Ride details, including pickup location, destination, driver information, and trip duration, shall be securely stored for future reference and investigation if required.

The system shall continuously monitor ride status and detect abnormal ride termination or communication failures whenever possible.

Safety Requirements

- Driver verification shall be mandatory before account activation.
- Vehicle documents shall be validated by the administrator.
- Emergency contact information shall be available to users.
- Trip records shall be securely stored.
- GPS tracking shall remain active throughout the ride.
- Users shall receive notifications for important ride events.
- Unauthorized access to ride information shall be prevented.

5.3 Security Requirements

The Ride Booking System shall implement strong security mechanisms to protect sensitive user information, payment details, and communication between system components.

All user passwords shall be encrypted before being stored in the database using secure hashing algorithms. Authentication shall be implemented using secure login mechanisms such as JWT (JSON Web Token) or equivalent technologies.

All communication between the client application and server shall occur over HTTPS using SSL/TLS encryption to prevent unauthorized interception of data.

Role-Based Access Control (RBAC) shall be implemented to ensure that passengers, drivers, and administrators can access only the functionalities permitted to their respective roles.

The system shall validate all user inputs to prevent SQL Injection, Cross-Site Scripting (XSS), Cross-Site Request Forgery (CSRF), and other common web security vulnerabilities.

Payment transactions shall be processed only through trusted third-party payment gateways. Sensitive financial information shall never be stored directly within the Ride Booking System.

Security Requirements

- Passwords shall be encrypted before storage.
- HTTPS shall be mandatory for all communication.
- JWT authentication shall be implemented.
- Multi-factor authentication may be supported in future versions.
- SQL Injection attacks shall be prevented.
- XSS attacks shall be prevented.
- CSRF protection shall be implemented.
- Secure payment gateways shall be used.
- User sessions shall automatically expire after inactivity.
- Administrator activities shall be logged for auditing purposes.

5.4 Software Quality Attributes

The Ride Booking System shall satisfy several software quality attributes to ensure long-term maintainability and user satisfaction.

Reliability

The application shall perform its intended functions consistently without unexpected failures. The system should recover gracefully from temporary errors and continue normal operation whenever possible.

Availability

The Ride Booking System shall remain available **24 hours a day, 7 days a week**, except during scheduled maintenance.

Scalability

The system architecture shall support future expansion by allowing additional users, drivers, cities, and services without requiring major redesign.

Maintainability

The software shall follow modular architecture and coding standards to simplify future enhancements, debugging, and maintenance.

Portability

The application shall operate on multiple operating systems and modern web browsers with minimal modifications.

Usability

The interface shall be intuitive and user-friendly, enabling first-time users to understand the application without extensive training.

Flexibility

The application shall support future integration with additional services such as ride sharing, AI-based route optimization, multilingual support, and electric vehicle booking.

Interoperability

The system shall communicate effectively with external services including Google Maps, payment gateways, SMS services, and email providers.

Robustness

The system shall continue operating correctly even when invalid inputs, network interruptions, or unexpected errors occur.

Testability

All modules shall be designed to support unit testing, integration testing, system testing, and user acceptance testing.

5.5 Business Rules

Business rules define the operational policies that govern the Ride Booking System. These rules ensure consistency, fairness, and compliance with organizational requirements.

The following business rules shall be enforced by the system:

1. Every passenger must register before booking a ride.
2. Every driver must complete document verification before accepting ride requests.
3. A passenger shall have only one active ride at a time.
4. A driver shall not accept multiple rides simultaneously.
5. Ride fare shall be calculated based on travel distance, ride category, estimated travel time, and applicable pricing rules.
6. Cancellation charges may be applied according to predefined cancellation policies.
7. Payments shall be processed only after ride completion unless advance payment is required.
8. Digital invoices shall be generated automatically after successful payment.
9. Ratings and reviews shall be submitted only after ride completion.
10. Administrators shall have authority to suspend fraudulent users or drivers.
11. Driver earnings shall be calculated automatically after deducting platform commissions, if applicable.
12. All ride records shall be maintained for future reporting, analytics, and dispute resolution.
13. The system shall maintain logs of significant administrative actions for auditing purposes.
14. Promotional offers and discounts shall be applied according to predefined business policies.
15. Future pricing strategies, such as surge pricing, may be incorporated without affecting existing booking functionality.

6. Other Requirements

This section specifies additional requirements necessary for the successful implementation, deployment, and maintenance of the Ride Booking System. These requirements include database specifications, deployment environment, and future enhancements that support the overall functionality of the system.

6.1 Database Requirements

The Ride Booking System shall use **MySQL** as the backend relational database to store and manage all application data securely.

The database shall maintain information related to:

- User Details
- Driver Details
- Vehicle Information
- Ride Bookings
- Payment Records
- Ratings and Reviews
- Notifications
- Administrator Details

The database shall maintain data integrity using **primary keys**, **foreign keys**, and appropriate constraints. Passwords shall be stored in encrypted format, and duplicate email addresses or mobile numbers shall not be allowed.

6.2 Deployment Requirements

The Ride Booking System shall be deployed on a secure server capable of handling multiple users simultaneously.

The deployment environment includes:

- Operating System: Windows or Linux
- Backend Framework: Spring Boot (Java)
- Database: MySQL
- Web Server: Apache Tomcat
- Internet Connectivity
- HTTPS for secure communication

The application shall support future updates and maintenance with minimal downtime.

6.3 Backup and Recovery

The system shall maintain regular backups of all important data to prevent data loss.

The backup process shall include:

- Daily database backup
- Secure storage of backup files
- Recovery of data in case of system failure

This ensures that user information, ride history, and payment records remain available even during unexpected failures.

6.4 Future Enhancements

The Ride Booking System is designed to support future improvements without major modifications to the existing architecture.

Possible future enhancements include:

- Ride Sharing (Carpool)
- AI-based Driver Assignment
- Dynamic Fare Pricing
- Voice-Based Ride Booking
- Multi-Language Support
- Electric Vehicle (EV) Booking
- AI Chatbot for Customer Support
- Loyalty and Rewards Program

These features can be integrated in future versions to improve user experience and system efficiency.

6.5 Summary

The additional requirements described in this section ensure that the Ride Booking System is reliable, maintainable, and scalable. Proper database management, secure deployment, regular backups, and support for future enhancements will help the system operate efficiently while meeting user and business requirements.

Appendix A: Glossary

The following glossary defines important terms, abbreviations, and acronyms used throughout this Software Requirements Specification (SRS).

Term / Acronym	Description
Admin	Administrator responsible for managing users, drivers, and system operations.
API	Application Programming Interface used for communication between software components.
Driver	Registered user who provides transportation services through the platform.
GPS	Global Positioning System used for determining and tracking locations.
HTTPS	Hypertext Transfer Protocol Secure for encrypted communication.
JWT	JSON Web Token used for secure user authentication.
MySQL	Relational Database Management System used to store application data.
Passenger (Rider)	User who books rides using the application.
Ride	A transportation service booked by a passenger and accepted by a driver.

Ride Booking System	The software application developed for booking, tracking, and managing rides.
SRS	Software Requirements Specification document.
UI	User Interface through which users interact with the system.
UML	Unified Modeling Language used for software design diagrams.
OTP	One-Time Password used for user verification.

Appendix B: Analysis Models

<Optionally, include any pertinent analysis models, such as data flow diagrams, class diagrams, state-transition diagrams, or entity-relationship diagrams.>

Appendix C: To Be Determined List

<Collect a numbered list of the TBD (to be determined) references that remain in the SRS so they can be tracked to closure.>